Aufgabenblatt 10

Effiziente Suche mit Hash Sets

Abgabe (bis 21.07.2025 23:59 Uhr)

Die folgenden Dateien werden für die Bepunktung berücksichtigt:

Geforderte Dateien:	
`Blatt10/src/Position.java`	Aufgabe 3.2
`Blatt10/src/Board.java`	Aufgabe 3.3
`Blatt10/src/PartialSolution.java`	sowie
`Blatt10/src/RicochetRobots.java`	Aufgabe 3.4

Als Abgabe wird jeweils nur die letzte Version im main branch in git gewertet.

Aufgabe 1: Hashing (Tutorium)

1.1
1. Wie funktioniert Hashing und was sind die Vorteile?
2. Was sind Kriterien für eine gute Hashfunktion?
3. Wo wird Hashing angewandt?

Lösung

1. Beim Hashing wird für ein meist großes oder stark parametrisiertes Datenobjekt (z.B. Daten eines Studenten) oder dessen Identifizierungsschlüssel (z.B. Matrikelnummer) der sogenannte Hashwert berechnet (meist vom Typ `int` oder `String`). Dieser lässt sich einfacher und schneller mit anderen Hashwerten vergleichen als das Datenobjekt oder der Schlüssel selbst. In Hashtabellen bildet die Hashfunktion einen Schlüssel direkt auf eine Position in der Tabelle ab, was (bei wenig Kollisionen) **konstante Zugriffszeit** auf das Datenobjekt erlaubt, im Vergleich zu linearer Zugriffszeit bei Listen.

2.
   - **Surjektivität**: Jeder Hashwert im Wertebereich der Hashfunktion soll auch mindestens von einem möglichen Datenobjekt berechnet werden können.
   - **Chaos**: Hashwerte für ähnliche Eingabewerte (z.B. nur 1 Bit Unterschied) sollten möglichst stark voneinander abweichen (z.B. für Fehlererkennung).
   - **Gleichverteilung**: Alle Hashwerte sollen von annähernd gleich vielen Eingabewerten berechenbar sein, sodass selten Überschneidungen (**Kollisionen**) auftreten.
   - **Effizienz**: Hashwerte sollten wenig Speicher und Berechnungszeit benötigen.
   - **In Java**: Auszug aus JAVA API 6 Object.hashCode(); The general contract of hashCode is:

- Whenever it is invoked on the same object more than once during an execution of a Java application, the `hashCode` method must consistently return the same integer, provided no information used in `equals` comparisons on the object is modified. This integer need not remain consistent from one execution of an application to another execution of the same application.
- If two objects are equal according to the `equals(Object)` method, then calling the `hashCode` method on each of the two objects must produce the same integer result.
- It is not required that if two objects are unequal according to the `equals(java.lang.Object)` method, then calling the `hashCode` method on each of the two objects must produce distinct integer results. However, the programmer should be aware that producing distinct integer results for unequal objects may improve the performance of hashtables.

3. Beispiele:

- **Hashtabellen**: Hier werden Schlüssel von Datenobjekten auf Positionen innerhalb einer Tabelle abgebildet, was (meist) konstante Suchzeit erlaubt. Der Wertebereich der Hashfunktion sollte der Kapazität der Tabelle entsprechen!
- **Prüfsummen**: Minimale Unterschiede in Dateien können an großen Unterschieden der Daten-Hashwerte erkannt werden. Diese Unterschiede können durch Übertragungsfehler oder absichtliche Manipulation (bis heute nur begrenzt feststellbar) auftreten.
- **Kryptologie**: One-Way-Hash Functions sind praktisch nicht invertierbar; d.h., es ist nahezu unmöglich, die Daten bei gegebenem Hashwert zu berechnen. Bei Passwörtern muss daher nur der Hashwert, nicht das Passwort selbst gespeichert werden. Die Hashfunktion SHA-512 liefert Hashwerte der Länge 512 Bit , was $1.34 \cdot 10^{154}$ möglichen Werten entspricht (das ist mehr als die Zahl der Wasserstoffatome im Universum, als Vergleich).

1.2 Sei k die Stelle eines Buchstaben im Alphabet. Betrachten Sie nun Hashing mit folgender Hashfunktion $h(k) = 11k \mod 7$. Fügen Sie damit nacheinander die Buchstaben des Worts ALGODAT in die Hashtabelle ein. Verwenden Sie dabei zur Kollisionsauflösung lineares Sondieren mit Inkrement 1 (d.h. $s(n) = n$).

0	1	2	3	4	5	6

Lösung

Werte:

A	L	G	O	D	T
11%7=4	132%7=6	77%7=0	165%7=4	44%7=2	220%7=3

Hashtabelle

0	1	2	3	4	5	6
G	A	D	T	A	O	L

Aufgabe 2: Laufzeitanalyse (Tutorium)

2.1 Geben Sie möglichst genau die Wachstumsordnung für den folgenden Code an. Begründen Sie Ihre Herleitung der Laufzeit am Code. Gehen Sie davon aus, dass der Graph N Knoten enthält und E Kanten. `solved` ist hierbei ein `boolean` Array der Länge N.

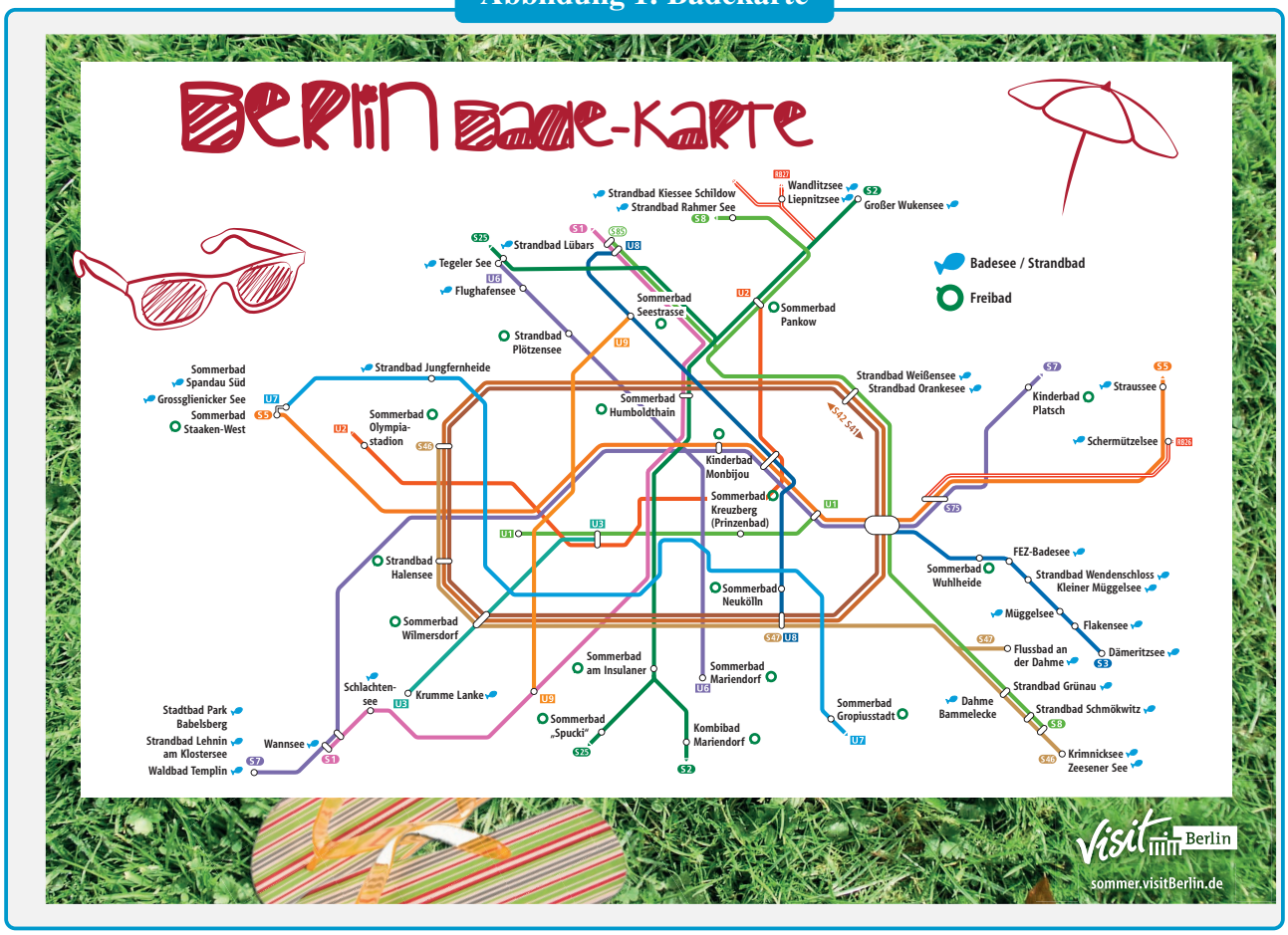
```
public void mystery(Graph G, int v, boolean[] solved) {
    solved[v] = true;
    for (int w : G.adj(v)) {
        if (!solved[w]) {
            mystery(G, w, solved);
        }
    }
}
```

Was macht der Code? Kennen Sie diesen Algorithmus?

Lösung

Das ist die Tiefensuche rekursiv.

Abbildung 1: Badekarte



2.2 Erklären Sie an der Karte[1] in Abbildung 1, welche unterschiedlichen Konzepte von Graphen Sie kennengelernt haben, welche Algorithmen auf diesen Graphen Sie kennen und welche Arten von Problemen diese lösen.

Lösung

Grahen: gerichtet, ungerichtet, gewichtet, ungewichtet, Flussgraphen
Graphen-Algorithmen: BFS, DFS, Prim, Kruskal, topologische Sortierung, Dijkstra, Bellmann-Ford, Edmonds-Karp...
Probleme:

- Finde Seebad/Freibad.
- Minimale Netzdarstellung, die alle Bademöglichkeiten beinhaltet - MST
- Kürzester Weg!
- Auslastung auf den Strecken - Flussgraphen.

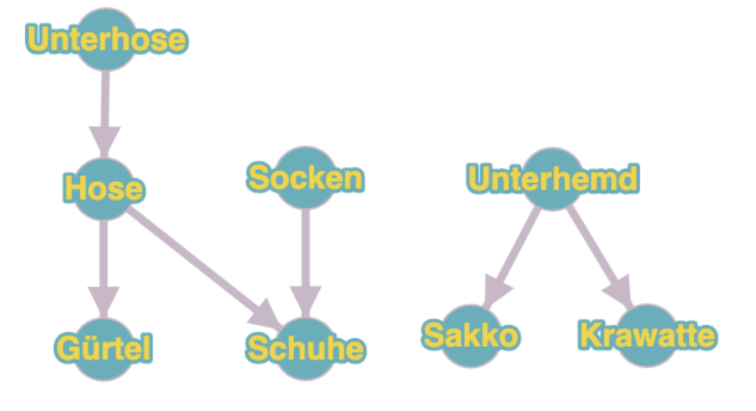
2.3 **(Optional)** Erstellen sie für folgende Kleidungsstücke Knoten und Kanten in einem gerichteten Graphen, wobei eine Kante bedeutet, dass der Startknoten vor dem Endknoten angezogen werden muss: Unterhemd (0), Socken (1), Krawatte (2), Gürtel (3), Schuhe (4), Hose (5), Sakko (6), Unterhose (7).

Geben sie anschließend eine Topologische Sortierung und eine Beispielreihenfolge für das Anziehen an, indem sie den folgenden Code per Hand simulieren. Wie muss dafür die Methode `public void mystery` angepasst werden? Für welche Graphen funktioniert der Algorithmus?

```
public void topologicalSortClothing(Graph g){
    Stack<Integer> stack = new Stack<Integer>();
    boolean visited[] = new boolean[G.V()];
    for (int i = 0; i < G.V(); i++){
        visited[i] = false;
    }
    for (int i = 0; i < V; i++){
        if (!visited[i]){
            mystery(G, i, visited, stack);
        }
    }
    while (!stack.isEmpty()){
        System.out.print(stack.pop() + " ");
    }
}
```

[1] https://www.visitberlin.de/de/blog/berlin-bade-karte

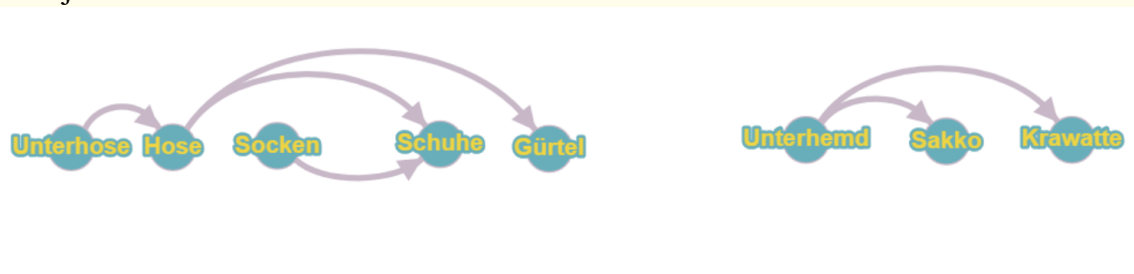
Lösung



Damit der Code funktioniert, muss in der `mystery`-Methode der Stack als Parameter übergeben werden und der Knoten `v` auf den Stack gepusht werden, sobald alle seine adjazenten Knoten besucht wurden.

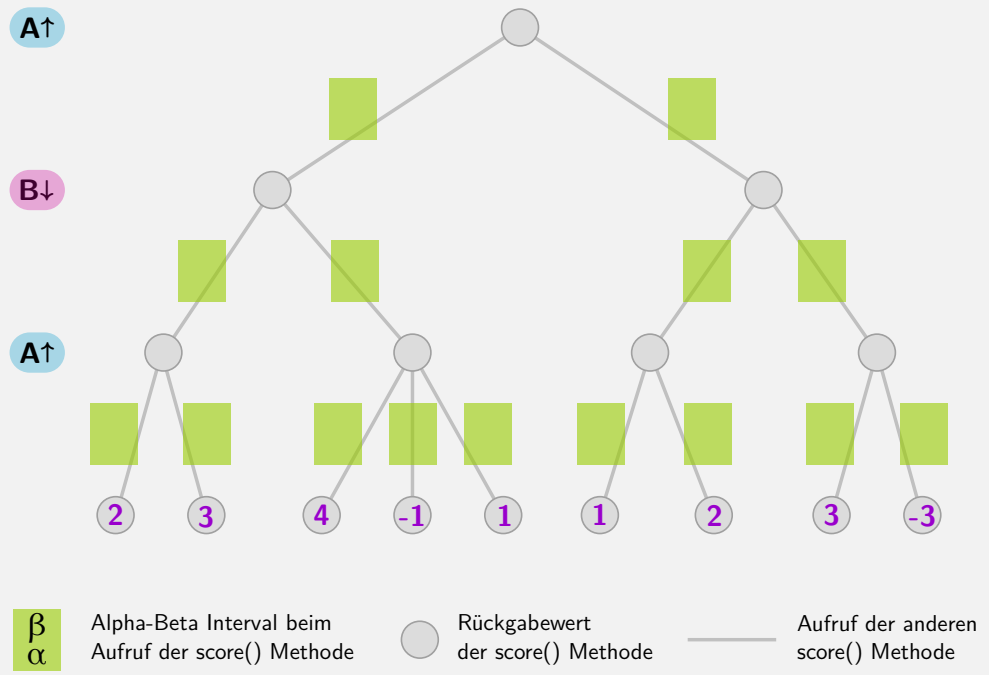
```
public void mystery(Graph G, int v, boolean solved[], Stack<Integer> stack) {
    solved[v] = true;
    for (int w : G.adj(v)) {
        if (!solved[w]) {
            mystery(G, w);
        }
    }
    stack.push(v);
}
```

Der Algorithmus funktioniert nur für DAGs (directed acyclic graphs).
Der Algorithmus generiert natürlich je nachdem wie der DFS verläuft, verschiedene Beispielreihenfolgen, dazu könnte man sich auch zufällig den Knoten raus suchen bei dem man den DFS startet, anstatt von 0 bis V zu iterieren, eine Beispielreihenfolge: Socken, Unterhose, Hose, Schuhe, Gürtel, Unterhemd, Sakko, Krawatte
Erläutert außerdem dazu die grafische Lösung, indem man die Knoten linear so anordnet, dass jede Kante nur noch von links nach rechts führt:



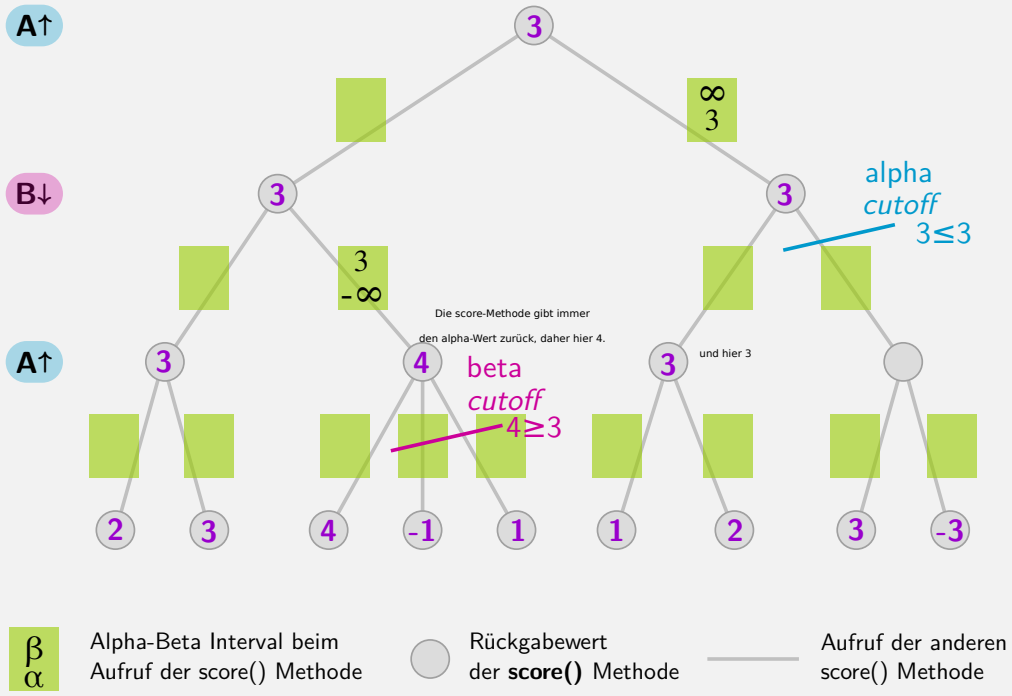
2.4 **(Optional)** Gehen Sie die Handsimulation der Alpha-Beta-Suche in Abbildung 2 schrittweise durch. Markieren Sie ggf. α- und β- Cutoffs.

Abbildung 2: Alpha-Beta Suche- Handsimulation



Lösung

Abbildung 2: Alpha-Beta Suche Lösung



Siehe score-Methode VL 5, Folie 15 und Folie 17. Auch hilfreich: `https://www.`

`youtube.com/watch?v=xBXHtz4Gbdo`. Unsere alpha und beta Intervalle, die erwartet werden, sind immer die, die von einem Knoten zu einem anderen nach UNTEN gehen. Die Werte in den Knoten werden erwartet, wie sie von der score-Methode am Ende zurückgegeben werden. Diese gibt α zurück. Siehe dazu auch die Lösung im entsprechenden Tutorium.

Aufgabe 3: Für die Hausaufgaben wird keine Lösung bereitgestellt.

Was Sie nach diesem Blatt wissen sollten:

- Was Hashing ist
- Was perfektes Hashing ist und was sind Kollisionen sind
- Was Strategien zur Kollisionsauflösung sind und wie diese genau funktionieren
- Was die Besonderheiten beim Einfügen und Löschen von Schlüsseln sind und was lazy und eager deletion sind
- Was der Belegungsfaktor ist und welchen Einfluss er auf die Laufzeit hat
- Wie Hashing mit Java-Objekten funktioniert